

Arcade 2D

<https://github.com/asitsy/Arcade2D>

Zaawansowane Programowanie Obiektowe – Laboratorium

Imię i nazwisko: Anastasiia Tsyban

Numer albumu: 31024

Tryb studiów: I stopnia, Stacjonarne

Kierunek: Informatyka

Semestr: IV

Data rozpoczęcia: 14.05.2026

1. Opis projektu

Projekt jest grą 2D typu Pac-Man zrealizowaną w MonoGame.

Gracz steruje postacią Player, zbiera Pellet (mam zwykle + Powerpellet, SpeedPellet),

PowerPellet nadaje Ghosts zamrożenie na 5 sekund.

SpeedPellet nadaje zwiększenie prędkości Player na 5 sekund.

Unika Ghost i przechodzi między stanami Menu, Gameplay, Victory i GameOver.

Gracz ma 1 szansę na kolizję z Ghost'em, po tym 5 sekund zamrożenia i gracz wystartuje z początkowego miejsca gry.

Gra zapisuje ostatni wynik gry za pomocą async

2. Struktura projektu

Arcade2D/Game1.cs - Główna klasa silnika MonoGame. Odpowiada za cykl życia aplikacji: inicjalizację okna gry, wczytywanie zasobów (grafik, czcionek), główną pętlę aktualizacji logiki (Update) oraz renderowanie klatek na ekranie (Draw). Zarządza również przełączaniem stanów (State Machine) oraz instancjami menedżerów.

Arcade2D/Program.cs - Punkt wejścia do aplikacji (ang. entry point). Zawiera statyczną metodę Main, która inicjalizuje środowisko i uruchamia główną pętlę gry.

Arcade2D /Managers:

CollisionManager.cs - Menedżer kolizji. Analizuje i przetwarza interakcje fizyczne między obiektami na planszy. Sprawdza m.in., czy gracz uderzył w ścianę, czy dotknął ducha (co skutkuje utratą życia) oraz czy zebrał punkty lub bonusy (np. PowerPellet).

EntityManager.cs - System zarządzania bytami. Przechowuje listę wszystkich aktywnych obiektów na planszy. Odpowiada za ich dodawanie, usuwanie w trakcie gry (np. znikanie zebranych punktów) oraz zbiorowe wywoływanie dla nich metod Update i Draw.

ScoreManager.cs - Menedżer wyniku. Odpowiada za zliczanie punktów zdobytych przez gracza podczas rozgrywki oraz asynchroniczne zapisywanie i odczytywanie ostatniego wyniku (Last Score) z zewnętrznego pliku tekstowego (lastscore.txt).

Arcade2D/Utils:

ColorPalette.cs - Statyczna klasa przechowująca definicje niestandardowych kolorów używanych w interfejsie i grze. Zapewnia to spójność wizualną całego projektu i ułatwia ewentualną zmianę palety barw.

MapLoader.cs - System generowania poziomów. Wczytuje układ mapy z pliku tekstowego (level1.txt) i na podstawie odpowiednich znaków (np. '1' to ściana, '0' to puste pole) dynamicznie tworzy (instancjonuje) obiekty Wall, Pellet, Ghost oraz Player.

States:

State.cs - Abstrakcyjna klasa bazowa dla wszystkich ekranów. Wymusza na klasach pochodnych implementację podstawowych metod Update i Draw.

MenuState.cs - Ekran startowy. Wyświetla menu główne gry, tytuł, informacje o autorze oraz przycisk pozwalający na rozpoczęcie nowej rozgrywki.

VictoryState.cs - Właściwy stan rozgrywki. To tutaj toczy się gra: gracz porusza się po labiryncie, duchy używają sztucznej inteligencji, sprawdzane są kolizje, a menedżery aktualizują stan punktów i życie.

GameplayState.cs - Ekran wyświetlany po pomyślnym ukończeniu poziomu (zjedzeniu wszystkich kulek na planszy). Wyświetla gratulacje i końcowy wynik.

GameOverState.cs - Ekran porażki. Wyświetlany, gdy gracz straci wszystkie życia. Pokazuje podsumowanie punktowe i daje możliwość restartu.

/Entities:

Entity.cs - Abstrakcyjna klasa bazowa dla każdego bytu na mapie. Definiuje wspólne właściwości, takie jak pozycja 2D (wektor) oraz przypisana tekstura.

Wall.cs - Reprezentuje nieprzekraczalną przeszkodę architektoniczną w labiryncie.

Pellet.cs - Zwykła kulka / punkt. Podstawowy cel gry, znikający po zebraniu przez gracza i powiększający jego wynik.

PowerPellet.cs - Specjalny bonus (żółta kulka). Zebranie go tymczasowo zamraża przeciwników i daje graczowi przewagę.

SpeedPellet.cs - Specjalny bonus (różowa kulka), który po zebraniu na kilka sekund zwiększa prędkość poruszania się gracza, ułatwiając ucieczkę przed duchami.

Ghost.cs - Przeciwnik (Duch). Klasa zawierająca własną logikę poruszania się (sztuczną inteligencję), obsługę skrzyżowań w labiryncie oraz system animacji. Jeden czerwony zawsze idzie za graczem, inne sa szukają wejścia na korytarzach.

Player.cs - Klasa postaci sterowanej przez gracza. Obejmuje obsługę wejścia (Input z klawiatury), logikę prędkości, płynne animacje zmiany kierunku oraz ograniczenia ruchu związane ze ścianami.

/Interfaces:

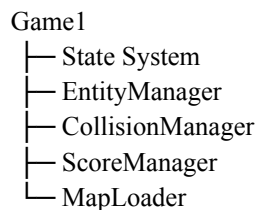
IRenderable.cs - Interfejs dla obiektów, które posiadają swoją reprezentację graficzną i implementują metodę Draw (rysowanie na ekranie).

ICollidable.cs - Interfejs dla obiektów, z którymi można wejść w fizyczną interakcję. Wymaga definicji obszaru kolizji (np. właściwość Bounds zwracająca obiekt Rectangle).

IUpdatable.cs - Interfejs dla bytów i menedżerów, których stan musi być odświeżany w każdej klatce animacji (implementacja metody Update).

3. Architektura

Architektura opiera się na State Pattern (State/MenuState/GameplayState/GameOverState/VictoryState), Entity System (Entity, Player, Ghost, Pellet, Wall), oraz Manager Pattern (EntityManager, CollisionManager, ScoreManager).



Game1.cs

Typy: Game1

Metody:

- protected Initialize()
- protected LoadContent()
- public ChangeState()
- public RestartGame()
- protected Update()
- protected Draw()

Arcade2D/Managers/CollisionManager.cs

Typy: CollisionManager

Metody:

- public CheckWallCollision()
- public IsCollidingWithWall()
- public UpdateGameplayCollisions()

Arcade2D/Managers/EntityManager.cs

Typy: EntityManager

Metody:

- public Add()
- public DestroyEntity()
- public Clear()
- public Update()
- public Draw()

Arcade2D/Managers/ScoreManager.cs

Typy: ScoreManager

Metody:

- public Reset()
- public AddScore()
- public LoadLastScore()
- public SaveLastScoreAsync()
- public UpdateLastScoreInMemory()

Arcade2D/Utils/ColorPalette.cs

Typy: ColorPalette

[Arcade2D/Utils/MapLoader.cs](#)

Typy: MapLoader

Metody:

- public LoadSpriteSheetWithChromaKey()
- public BuildMap()

[Arcade2D/Utils/TextureExtensions.cs](#)

[Arcade2D/States/GameOverState.cs](#)

Typy: GameOverState

Metody:

- public Update()
- public Draw()

[Arcade2D/States/VictoryState.cs](#)

Typy: VictoryState

Metody:

- public Update()
- public Draw()

[Arcade2D/States/GameplayState.cs](#)

Typy: GameplayState

Metody:

- public ResetTimers()
- public Update()
- public Draw()

[Arcade2D/States/MenuState.cs](#)

Typy: MenuState

Metody:

- public Update()
- public Draw()
- private DrawBorder()

[Arcade2D/States/State.cs](#)

Typy: State

Metody:

- public Update()
- public Draw()

[Arcade2D/Entities/Wall.cs](#)

Typy: Wall

Metody:

- public Update()

[Arcade2D/Entities/Pellet.cs](#)

Typy: Pellet

Metody:

- public Update()

[Arcade2D/Entities/PowerPellet.cs](#)

Typy: PowerPellet

[Arcade2D/Entities/SpeedPellet.cs](#)

Typy: SpeedPellet

[Arcade2D/Entities/Ghost.cs](#)

Typy: Ghost

Metody:

- public Update()
- private ChooseNewDirection()
- public Update()
- public Draw()

[Arcade2D/Entities/Player.cs](#)

Typy: Player

Metody:

- public SetSpedUp()
- public ResetPosition()
- public Update()
- public Update()
- public Draw()

[Arcade2D/Entities/Entity.cs](#)

Typy: Entity

Metody:

- public Draw()
- public Update()

[Arcade2D/Interfaces/IRenderable.cs](#)

Typy: IRenderable

[Arcade2D/Interfaces/ICollidable.cs](#)

Typy: ICollidable

Arcade2D/Interfaces/ IUpdatable.cs

Typy: IUpdatable

Ghost AI

Klasa Ghost implementuje ruch po siatce. Duch utrzymuje aktualny kierunek, zbiór możliwych kierunków oraz wybiera nowe kierunki na skrzyżowaniach. W kodzie występuje logika losowa (Random) oraz wariant ducha śledzący pozycję gracza częściej niż pozostałe.

Mapowanie

MapLoader buduje mapę na podstawie danych tekstowych. Tworzy obiekty Wall, Pellet, PowerPellet, SpeedPellet, Ghost i Player i rejestruje je w EntityManager.

Pętla gry

Program -> Game1 -> Initialize -> LoadContent -> Utworzenie stanów -> RestartGame -> BuildMap -> Main Loop. W pętli wykonywane są Update() i Draw().

Refaktoryzacja oraz części które dodam później:

- chciałabym dodać dźwięk (przez AudioManager)
- wprowadzenie ResourceManager.
- rozdzielenie Game1 na mniejsze komponenty.

Literatura oraz linki które były dla mnie pomocne:

<https://www.sprites-resource.com/arcade/pacman/> - tutaj miałam spritesheet Pacman & Ghosts

<https://www.deepl.com/pl/translator>

<https://docs.monogame.net/> - Oficjalna dokumentacja MonoGame Framework

<https://learn.microsoft.com/pl-pl/dotnet/csharp/asynchronous-programming/> - Artykuły techniczne, które posłużyły do poprawnej implementacji asynchronicznego zapisu plików (File.WriteAllTextAsync) bez blokowania głównego wątku renderującego.

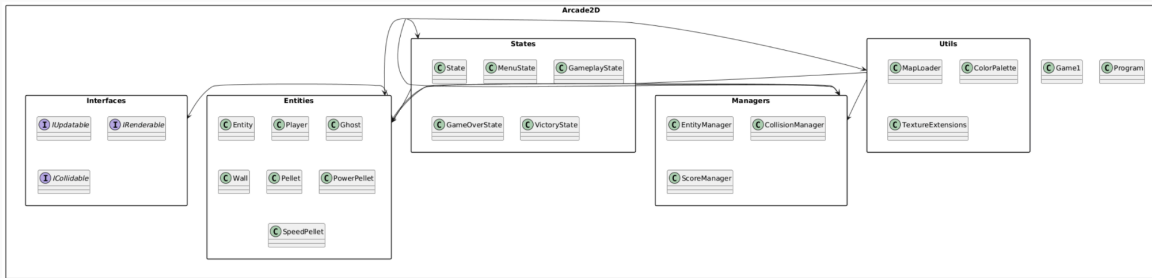
<http://rbwhitaker.wikidot.com/monogame-tutorials> - MonoGame Tutorials – RB. Whitaker Zbiór praktycznych poradników dotyczących detekcji kolizji 2D na podstawie otaczających prostokątów (Bounding Boxes) oraz transformacji tekstur.

Książki i publikacje

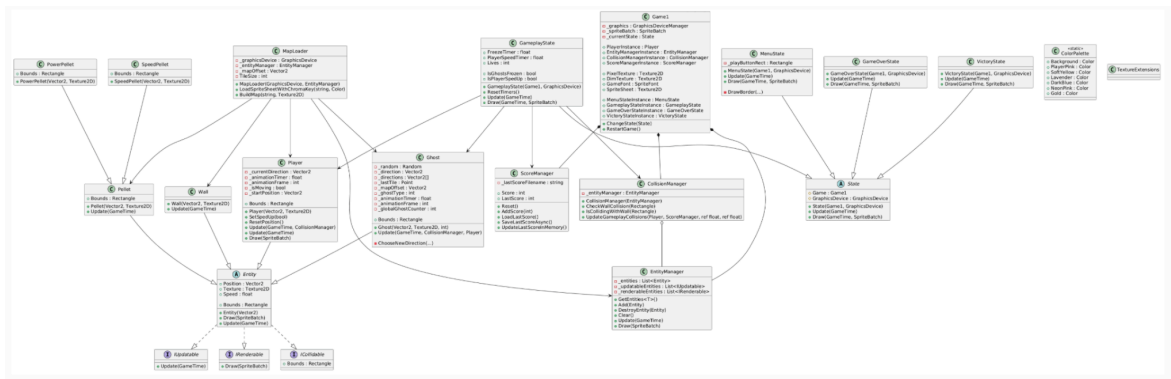
1. Nystrom, R. (2014). Wzorce projektowe w grach. Pozycja kluczowa dla zrozumienia architektury pętli gry (Game Loop), systemu aktualizacji obiektów (Update Method) oraz implementacji maszyny stanów (State Pattern) użytej do zarządzania ekranami gry.
2. Troelsen, A., Japikse, P. (2022). Język C# 10 i platforma .NET 6. Wydawnictwo Helion. Podręcznik wykorzystany do pogłębienia wiedzy z zakresu programowania zorientowanego obiektowo (OOP), implementacji interfejsów oraz mechanizmów wielowątkowości i asynchroniczności (Task, async/await).
3. Miles, R. (2016). Begin More C# Development with MonoGame. Apress. Publikacja pomocna przy projektowaniu struktury encji (Entities) oraz obsługi wejścia (klawiatura i mysz) w środowisku MonoGame.

Diagramy UML

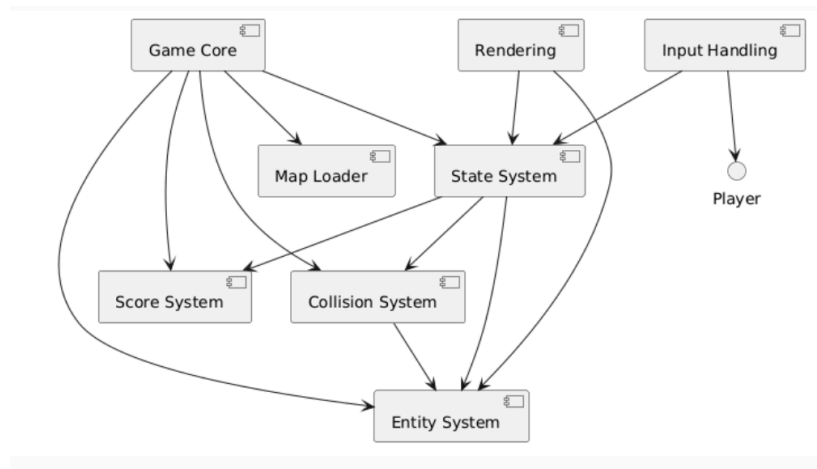
Package Diagram



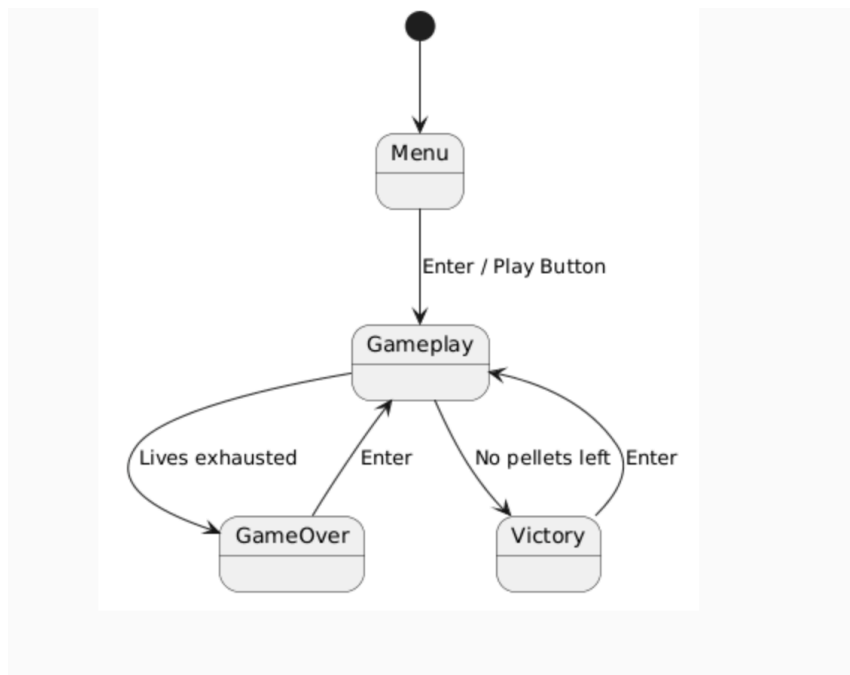
Complete Class Diagram



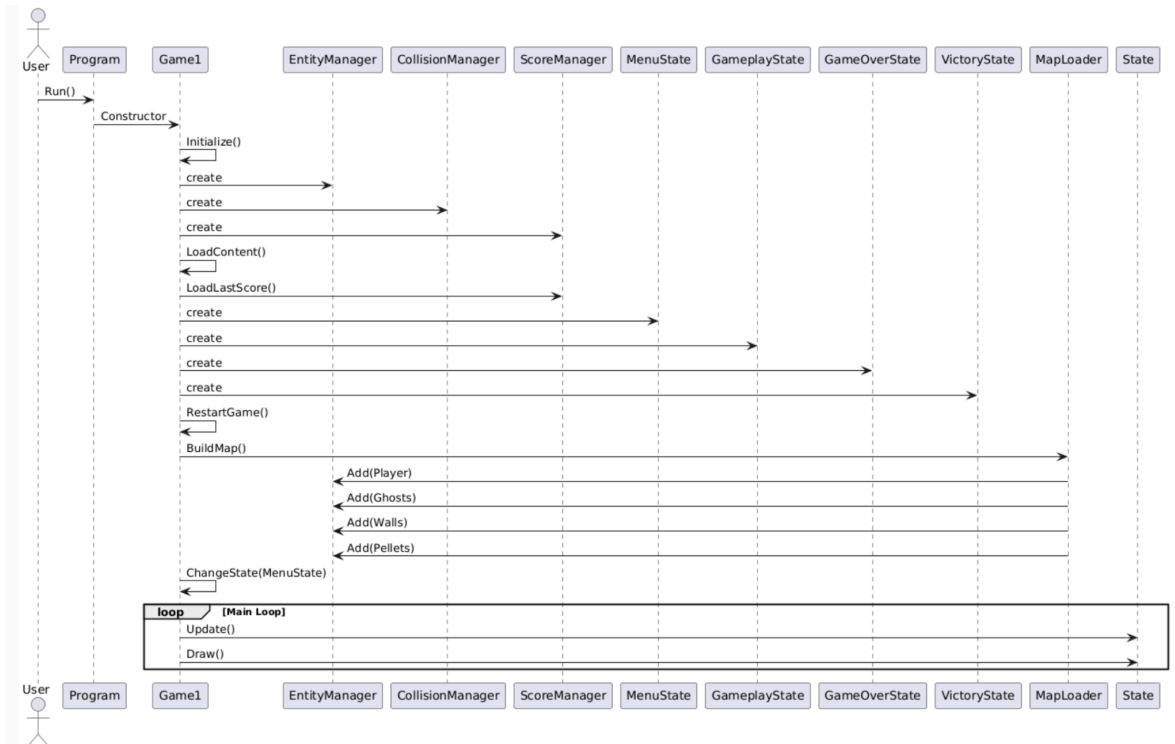
Component Diagram



State Diagram



Sequence Diagram — Game Startup



Sequence Diagram — Gameplay Update Cycle

